

Chapter Overview

Chapter 09 covers caching in ShopVerse: caching fundamentals, cache patterns, Redis as distributed cache, Spring Cache abstraction, Caffeine local cache, cache-aside vs write-through vs write-behind, cache invalidation strategies, cache warming, TTL design, and cache stampede prevention.

Sub-Chapter	Topic	Key Class / Config
09-01	Caching Fundamentals & Why Cache	Cache hit/miss, latency hierarchy, cache ratio metrics
09-02	Cache Patterns	Cache-aside, write-through, write-behind, read-through, refresh-ahead
09-03	Redis as Distributed Cache	RedisTemplate, Spring Cache with Redis, serialisation
09-04	Spring Cache Abstraction	@Cacheable, @CachePut, @CacheEvict, @Caching, CacheManager
09-05	Caffeine – Local In-Process Cache	CaffeineCache, expireAfterWrite, maximumSize, stats
09-06	Multi-Level Cache (L1 + L2)	Caffeine (L1) + Redis (L2) + DB fallback
09-07	Cache Invalidation Strategies	TTL, explicit eviction, event-driven, CDC with Debezium
09-08	Cache Stampede Prevention	Lock-based, probabilistic early refresh, background refresh
09-09	Cache Warming Strategies	ApplicationReadyEvent, scheduled warming, predictive prefetch
09-10	TTL Design Guidelines	Short-lived vs long-lived data, TTL per resource type
09-11	Cache Metrics & Monitoring	Hit ratio, eviction rate, memory usage, Micrometer
09-12	Redis Memory Management	maxmemory policies, key expiry, memory analysis

Latency Hierarchy (approximate)

Storage Tier	Typical Latency	IOPS	ShopVerse Layer
CPU L1 Cache	~0.5 ns	—	JVM JIT compiled code
CPU L2/L3 Cache	~5–20 ns	—	JVM hot objects
RAM / JVM Heap	~100 ns	—	Caffeine in-process cache
Redis (local)	~0.1–1 ms	>100k	Redis cache (product, session, cart)
Redis (network)	~1–5 ms	>50k	Redis over LAN in same AZ
SSD (NVMe)	~100 µs	100k+	Postgres WAL, index access
PostgreSQL query (cached)	~1–10 ms	~10k	PgBouncer pool hit
PostgreSQL query (disk)	~10–100 ms	~1k	Cold query, no cache
External API	~50–500 ms	—	Payment gateway, email service

Cache Hit Ratio

Hit Ratio	Meaning	Action
> 90%	Excellent – most reads served from cache	Monitor; maintain TTL strategy
70–90%	Good – most cache, some DB	Review TTL; check eviction rate
50–70%	Fair – cache helping but not enough	Increase maxmemory; review key design
< 50%	Poor – cache not effective	Review access patterns; consider pre-warming

Five Cache Patterns Comparison

Pattern	Read	Write	Consistency	ShopVerse Usage
Cache-aside (lazy)	App: check cache, miss? load from DB	App: write to DB, evict cache	Eventual (stale TTL possible)	Product category, user profile
Read-through	Cache: miss? cache load from DB	App: write to DB	Eventual (cache owns reading)	Transactional cache layer
Write-through	App writes to cache; cache writes to DB	App writes to DB and DB always sync	Strong sync	Critical data (cart, inventory)
Write-behind (async)	Read from cache	App writes to cache; async flush to DB	Eventual (risk of data loss)	High-write counters (views, likes)
Refresh-ahead	Cache proactively refreshes before TTL	None	Near-real-time	Popular product pages

Cache-Aside (Most Common in ShopVerse)

```
// Cache-aside pattern for ProductService
public Optional findById(long id) {
    String key = "product:" + id;
    // 1. Check cache
    Product cached = (Product) redisTemplate.opsForValue().get(key);
    if (cached != null) return Optional.of(cached); // cache HIT
    // 2. Cache MISS - load from DB
    Optional fromDb = productRepo.findById(id);
    // 3. Populate cache
    fromDb.ifPresent(p -> redisTemplate.opsForValue()
        .set(key, p, Duration.ofMinutes(60)));
    return fromDb;
}

// On update: invalidate cache
public Product update(long id, ProductUpdateRequest req) {
    Product updated = productRepo.save(mapper.toEntity(req));
    redisTemplate.delete("product:" + id); // explicit eviction
    return updated;
}
```

RedisTemplate Configuration

```
@Configuration public class RedisConfig {
    @Bean public RedisTemplate redisTemplate(RedisConnectionFactory f) {
        RedisTemplate t = new RedisTemplate<>();
        t.setConnectionFactory(f);
        t.setKeySerializer(new StringRedisSerializer());
        t.setValueSerializer(new GenericJackson2JsonRedisSerializer());
        t.setHashKeySerializer(new StringRedisSerializer());
        t.setHashValueSerializer(new GenericJackson2JsonRedisSerializer());
        return t;
    }
    @Bean public RedisCacheManager cacheManager(RedisConnectionFactory f) {
        RedisCacheConfiguration cfg = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(60))
            .serializeKeysWith(fromSerializer(new StringRedisSerializer()))
            .serializeValuesWith(fromSerializer(new GenericJackson2JsonRedisSerializer()))
            .disableCachingNullValues();
        return RedisCacheManager.builder(f)
            .cacheDefaults(cfg)
            .withCacheConfiguration("products", cfg.entryTtl(Duration.ofHours(2)))
            .withCacheConfiguration("categories", cfg.entryTtl(Duration.ofHours(24)))
            .withCacheConfiguration("sessions", cfg.entryTtl(Duration.ofHours(1)))
            .build();
    }
}
```

@Cacheable / @CachePut / @CacheEvict

Annotation	Behaviour	Returns	ShopVerse Use
@Cacheable(key)	Return cached; call method on miss; cache result or fresh	Cached or fresh	findById, findByCategory
@CachePut(key)	Always call method AND update cache	Fresh result	update, create (keep cache current)
@CacheEvict(key)	Evict entry on call	Method result	delete, status change
@CacheEvict(allEntries)	Evict ALL entries in cache	Method result	Admin: invalidate all products
@Caching({...})	Combine multiple operations	Method result	placeOrder: evict cart, put order

```

@Service public class ProductService {
    @Cacheable(value="products", key="#id",
    unless="#result == null") // don't cache null
    public Optional findById(long id) {
        return productRepo.findById(id);
    }
    @CachePut(value="products", key="#result.id") // cache the updated entity
    public Product update(long id, ProductUpdateRequest req) {
        return productRepo.save(mapper.apply(req, productRepo.findById(id).orElseThrow()));
    }
    @CacheEvict(value="products", key="#id") // remove on delete
    public void delete(long id) { productRepo.deleteById(id); }
    @CacheEvict(value="products", allEntries=true) // clear all on category change
    public void invalidateAll() {}
}

```

■ Spring Cache is a proxy-based AOP. Self-invocation (this.method()) bypasses the proxy and the cache annotation has no effect. Same caveat as @Transactional.

Caffeine Configuration

```
@Configuration @EnableCaching
public class CacheConfig {
    @Bean public CacheManager localCacheManager() {
        CaffeineCacheManager mgr = new CaffeineCacheManager();
        mgr.setCaffeine(Caffeine.newBuilder()
            .maximumSize(1000) // max 1000 entries
            .expireAfterWrite(5, MINUTES) // TTL: 5 min after write
            .expireAfterAccess(2, MINUTES) // evict if not accessed for 2 min
            .recordStats() // enable hit/miss stats
        );
        return mgr;
    }
}
```

Caffeine Eviction Policies

Policy	Annotation / Method	Behaviour	Use When
Size-based	maximumSize(n)	Evicts LRU when n exceeded (W-TinyLFU algorithm)	By default, evicting many unique keys
Time-to-live	expireAfterWrite(d)	Evict entry d after it was written	Stale-on-write semantics
Time-to-idle	expireAfterAccess(d)	Evict if not accessed for d	Evict rarely-used entries first
Reference-based	weakValues() / softValues()	Evict when GC reclaims referent	Memory pressure eviction
Custom	expireAfter(Expiry)	Per-entry expiry control	Different TTL per key

Caffeine vs Redis

Feature	Caffeine (L1)	Redis (L2)
Latency	~100 ns	~0.5–5 ms
Scope	Single JVM process	Shared across all pods
Max size	Bounded by JVM heap	Bounded by maxmemory setting
Persistence	No (in-memory only)	AOF/RDB optional
Clustering	No	Yes (Cluster/Sentinel)
ShopVerse use	Hot product data, config lookup	Product cache, sessions, cart, rate limit

Cache Hierarchy

Request: findProduct(id)

L1 (Caffeine) check – ~100ns – HIT? return immediately

L1 MISS -> L2 (Redis) check – ~1ms – HIT?

Populate L1 from Redis value; return

L2 MISS -> Database (PostgreSQL) query – ~5-20ms

Populate L2 (Redis) with TTL=60min

Populate L1 (Caffeine) with TTL=5min

Return result

```
@Service public class MultiLevelCacheProductService {
    private final Cache l1 = Caffeine.newBuilder()
        .maximumSize(500).expireAfterWrite(5, MINUTES).build();
    private final RedisTemplate l2;
    private final ProductRepository db;
    public Optional findById(long id) {
        // L1 check
        Product p = l1.getIfPresent(id);
        if (p != null) return Optional.of(p);
        // L2 check
        p = (Product) l2.opsForValue().get("product:"+id);
        if (p != null) { l1.put(id, p); return Optional.of(p); }
        // DB fallback
        Optional fromDb = db.findById(id);
        fromDb.ifPresent(prod -> {
            l2.opsForValue().set("product:"+id, prod, Duration.ofHours(1));
            l1.put(id, prod); });
        return fromDb;
    }
}
```

Strategy	Mechanism	Consistency	Complexity	ShopVerse
TTL expiry	Key expires after fixed duration	Eventual (stale within TTL)	Low	All cache keys have TTL
Explicit eviction	@CacheEvict on update/delete	Near-real-time	Low	Product, order status update
Event-driven (Spring)	@TransactionalEventListener (phase=AFTER_COMMIT)	Strong (after transaction commit)	Medium	OrderService -> evict cart
CDC (Debezium)	Postgres WAL stream -> Kafka -> cache invalidation	Near-real-time	High	Sync Elasticsearch index
Cache version key	Key includes version number	Strong version to invalidate	Medium	Bulk catalog invalidation

```
// Event-driven cache invalidation
@Component public class ProductCacheInvalidator {
    private final RedisTemplate redis;
    @TransactionalEventListener(phase=AFTER_COMMIT)
    public void onProductUpdated(ProductUpdatedEvent event) {
        redis.delete("product:" + event.productId());
        redis.delete("category-products:" + event.categoryId()); // also invalidate category list
    }
}
```

The Stampede Problem

When a popular key expires, N concurrent requests all miss the cache simultaneously. Each thread queries the DB and writes to cache, causing N database queries instead of 1.

Prevention Strategies

Strategy	Mechanism	Pros	Cons
Distributed lock (Redis SETNX)	On first request, requester rebuilds; others wait	Strongly consistent	Lock overhead; waiting requests blocked
Probabilistic early refresh	Probabilistically refresh before TTL expires	No lock needed; smooth	Some early refreshes wasted
Background async refresh	Before TTL: return stale, kick off async refresh	Never blocks user	Stale returned during refresh window
Jitter on TTL	Add random 0–10% to TTL to desynchronize expiry	Simple	Does not prevent stampede; reduces probability

```
// Background async refresh pattern
@Cacheable(value="products", key="#id")
public Product findById(long id) {
    return productRepo.findById(id).orElseThrow();
}

@Scheduled(fixedRate=50_000) // refresh every 50s (TTL=60s → 10s before expiry)
public void refreshPopularProducts() {
    popularProductIds().forEach(id -> {
        Product fresh = productRepo.findById(id).orElse(null);
        if (fresh != null) cacheManager.getCache("products").put(id, fresh);
    });
}

// Redis SETNX-based lock
public Product findWithLock(long id) {
    String lockKey = "lock:product:" + id;
    Boolean acquired = redis.opsForValue().setIfAbsent(lockKey, "1", Duration.ofSeconds(10));
    if (Boolean.TRUE.equals(acquired)) {
        try {
            Product p = productRepo.findById(id).orElseThrow();
            redis.opsForValue().set("product:"+id, p, Duration.ofMinutes(60));
            return p;
        } finally { redis.delete(lockKey); }
    }
    // Lock not acquired: return stale or wait then retry
    return (Product) redis.opsForValue().get("product:"+id);
}
```

```
// Warm cache on application start
@Component public class CacheWarmingService {
    @EventListener(ApplicationReadyEvent.class)
    public void warmUp() {
        log.info("Warming cache...");
        // Warm top-100 products
        productRepo.findTop100ByIsActiveTrueOrderBySalesCountDesc()
            .forEach(p -> redisTemplate.opsForValue()
                .set("product:"+p.getId(), p, Duration.ofHours(2)));
        // Warm all active categories
        categoryRepo.findAllActive().forEach(c ->
            redisTemplate.opsForValue().set("category:"+c.getId(), c, Duration.ofHours(24)));
        log.info("Cache warmed: products + categories");
    }
}
```

Warming Strategy	When	Data Warmed
On startup (Application Ready Event)	At app start	Top-N products, all categories, active promotions
Scheduled (off-peak)	Nightly at 2am	Full catalog, daily trending products
Predictive (based on traffic)	Before peak hours (9am)	Yesterday's top-viewed products
On-demand (first request)	Cache-aside pattern	Any product on first access

Resource	Recommended TTL	Rationale
Product detail	60 min	Changes infrequently; explicit eviction on update
Category list	24 hours	Rarely changes; admin manages categories
Active promotions	5 minutes	Time-sensitive; stale promo harmful to business
User session	60 minutes (sliding)	Reset on each request; expire idle sessions
Shopping cart	24 hours	User expects cart to persist across sessions
Rate limit window	60 seconds	Sliding window duration
Search results	5 minutes	Dynamic; catalog changes affect results
Product inventory count	1 minute	Changes frequently; over-caching causes oversell
User profile (non-PII)	30 minutes	Explicit eviction on profile update
JWT blacklist entry	Until token expiry	Must survive at least until original token expires
Static config (feature flags)	10 minutes	Refresh periodically; @RefreshScope

```
// Caffeine cache statistics
Cache cache = Caffeine.newBuilder()
    .maximumSize(1000).recordStats().build();
CacheStats stats = cache.stats();
log.info("Hit ratio: {} Load avg: {}ms Eviction count: {}",
    stats.hitRate(), stats.averageLoadPenalty()/1e6, stats.evictionCount());
// Micrometer integration - expose to Prometheus/Grafana
@Bean public CacheMetricsRegistrar cacheMetrics(MeterRegistry registry) {
    return new CacheMetricsRegistrar(registry, cacheManager());
}
```

Key Metrics to Monitor

Metric	Alert Threshold	Meaning
<code>cache.gets[result=hit]</code>	< 80% (alert)	Hit ratio dropping – cache not effective
<code>cache.gets[result=miss]</code>	> 20% (warn)	High miss rate – DB under pressure
<code>cache.evictions</code>	Sudden spike (alert)	Memory pressure; increase maxmemory or reduce TTL
<code>redis.memory.used</code>	> 80% maxmemory (alert)	Risk of eviction or OOM
<code>cache.load.duration</code>	p99 > 100ms (alert)	DB slow on cache miss; check query performance

Eviction Policies

Policy	Evicts	Use When
noeviction	Nothing (returns error on write)	Persistent storage; never for cache
allkeys-lru	LRU from all keys	Pure cache (all keys expendable)
volatile-lru	LRU from keys with TTL only	Mix of cache (TTL) + persistent (no TTL) keys
allkeys-lfu	LFU from all keys (Least Frequently Used)	Better than LRU for skewed access patterns
volatile-ttl	Key with shortest remaining TTL first	Evict most-expiring-soon keys
allkeys-random	Random from all keys	Rarely useful

```
# redis.conf / redis cluster config
maxmemory 2gb
maxmemory-policy allkeys-lru # pure cache: evict LRU
# Analysis commands
INFO memory # heap, RSS, fragmentation ratio
INFO stats # keyspace hits, misses, evicted_keys
redis-cli --hotkeys # top-accessed keys (useful for sizing)
redis-cli --bigkeys # find large-value keys (memory hogs)
DEBUG SLEEP 0 # test latency
```

Memory Analysis Workflow

Step	Command / Action	Purpose
1. Check usage	<code>INFO memory -> used_memory</code>	How much memory is used?
2. Check policy	<code>INFO server -> maxmemory-policy</code>	What eviction is active?
3. Check evicted	<code>INFO stats -> evicted_keys</code>	Has Redis been evicting keys?
4. Find big keys	<code>redis-cli --bigkeys</code>	Which keys use most memory?
5. Sample keys	<code>redis-cli --scan --pattern "product"</code>	How many keys per namespace?

■ Do

- ✓ Set TTL on ALL cache keys – use allkeys-lru eviction for pure cache deployments.
- ✓ Use cache-aside for read-heavy data; write-through for data where stale = risk.
- ✓ Multi-level cache: Caffeine (L1 5 min) + Redis (L2 60 min) + DB fallback.
- ✓ Use `@TransactionalEventListener(AFTER_COMMIT)` for cache eviction on data changes.
- ✓ Pre-warm popular data on `ApplicationReadyEvent` to avoid cold-start miss storm.
- ✓ Add jitter ($\pm 10\%$) to TTL values to desynchronise bulk expiry stampedes.
- ✓ Monitor hit ratio; alert if $< 80\%$. Hit ratio is the primary cache health metric.
- ✓ Use `unless="#result == null"` on `@Cacheable` to avoid caching null values.
- ✓ Size Caffeine `maximumSize` based on expected unique key count $\times$ avg value size vs heap.

■ Anti-Patterns

- No TTL on cache keys – unbounded memory growth; eventual eviction or OOM in Redis.
- Caching mutable session state in Caffeine – each pod has different view; inconsistency.
- Cache stampede: popular key expires; N threads hit DB simultaneously – use lock or jitter.
- Caching null values without explicit check – `unless="#result==null"` prevents this.
- Cache invalidation on transaction begin (not commit) – cache evicted but tx may rollback.
- Storing entire entity object when only a few fields needed – wastes cache memory.
- Using default Java serialisation for Redis values – fragile; versioning issues on deploy.
- allkeys-lru when mixing session (persistent) + cache (TTL) keys – evicts sessions too.

Sub-Chapter	Key Point	One-Liner
09-01 Hit Ratio	> 90%	Primary health metric; below 80% → review TTL/size
09-02 Cache-aside	Miss: load DB, set cache	Most common; explicit eviction on write
09-03 Redis config	GenericJackson2JsonRedisSerializer	Serialization; supports polymorphism
09-04 @CacheEvict	value="#result==null"	Never cache null; key SpEL for precision
09-05 Caffeine	maximumSize + recordStats()	Local L1; ~100ns; bounded by heap
09-06 Multi-level	L1 5min + L2 60min	Caffeine (fast) + Redis (shared across pods)
09-07 Invalidator	AFTER_COMMIT event	Evict only after DB transaction committed
09-08 Stampede	SETNX lock or jitter	Prevent N simultaneous DB hits on key expiry
09-09 Warming	ApplicationReadyEvent	Warm top-N on startup; avoid cold miss storm
09-10 TTL design	1min inventory, 24h categories	TTLs proportional to change frequency
09-11 Metrics	cache.gets[result=hit]	Monitor in Grafana; alert at < 80% hit ratio
09-12 Eviction	allkeys-lru	For pure cache; volatile-lru if mixing persistent keys

Interview Quick-Check

Question	Concise Answer
Cache-aside vs write-through?	Cache-aside: app manages load on miss. Write-through: every write goes to both cache and DB (avoid stale data).
Cache stampede?	Many concurrent misses on same expired key → all hit DB. Fix: SETNX lock, jitter TTL, or background refresh.
L1 (Caffeine) vs L2 (Redis)?	L1: per-JVM, ~100ns, not shared. L2: shared across all pods, ~1ms. L1 protects from Redis latency.
@CacheEvict vs @CachePut?	@CacheEvict: removes entry. @CachePut: always calls method AND updates cache (no stale wins).
When write-through?	When stale cache data causes business harm: inventory count, cart total, real-time pricing.